

Apunte de Clase: Programación Multicore – Modelos de Multithreading y Librerías de Hilos

Introducción

La mayoría de las aplicaciones modernas son multiproceso o multihilo. Un hilo (thread) es la unidad más pequeña de ejecución que puede ser planificada por el sistema operativo. A diferencia de un proceso, los hilos de un mismo proceso comparten el espacio de direcciones, los archivos abiertos, las señales y otros recursos, lo que los hace más livianos y eficientes.

En este apunte veremos:

- La diferencia entre proceso y hilo.
- Ventajas del multithreading.
- Modelos de relación entre hilos de usuario y hilos de kernel.
- Librerías estándar (Pthreads, Windows threads, Java threads).
- Ejemplos concretos en Linux (Pthreads y clone()).

1: Procesos vs. Hilos

1.1 Proceso tradicional (un solo hilo)

- Un proceso tiene un contador de programa (PC), una pila (stack), registros y un espacio de direcciones.
- Solo puede hacer una cosa a la vez.
- Si se bloquea (ej: esperando E/S), todo el proceso se detiene.

1.2 Proceso con múltiples hilos

- Un proceso puede contener varios hilos.
- Todos los hilos comparten:
 - Espacio de direcciones (código, datos globales, montón/heap).
 - Archivos abiertos.
 - Señales y manejadores de señales.
 - Directorio de trabajo actual, etc.
- Cada hilo tiene su propio:
 - Contador de programa (PC).
 - Conjunto de registros.
 - Pila (stack) independiente (para llamadas a funciones y variables locales).

1.3 ¿Por qué usar hilos?

Ventaja

Explicación

Capacidad de respuesta (Responsiveness)	Si un hilo se bloquea (por E/S), otros hilos pueden seguir ejecutándose. Esencial en interfaces gráficas (ej: un botón que sigue respondiendo mientras se guarda un archivo).
Compartir recursos (Resource sharing)	Los hilos comparten memoria y archivos de forma natural, sin necesidad de mecanismos complejos (como tuberías o memoria compartida explícita entre procesos).
Economía (Economy)	Crear un hilo es mucho más barato que crear un proceso. El cambio de contexto entre hilos del mismo proceso es más rápido (no hay cambio de tabla de páginas).
Escalabilidad (Scalability)	En sistemas multicore o multiprocesador, los hilos pueden ejecutarse en paralelo en distintos núcleos, acelerando la aplicación.

1.4 Aplicaciones típicas multihilo

- Procesadores de texto: Un hilo para la interfaz, otro para corrección ortográfica, otro para autoguardado.
- Servidores web: Un hilo por conexión entrante.
- Navegadores: Un hilo para la interfaz, otro para descarga, otro para renderizado.
- Kernels de sistemas operativos: Son intrínsecamente multihilo.

2: Paralelismo vs. Concurrencia

2.1 Definiciones claras

- Concurrencia (Concurrency): El sistema soporta más de una tarea en progreso, pero no necesariamente se ejecutan simultáneamente. En un único núcleo, el planificador alterna tareas (ilusión de paralelismo).
- Paralelismo (Parallelism): El sistema ejecuta más de una tarea realmente al mismo tiempo. Requiere múltiples núcleos o procesadores.

2.2 En un único núcleo (concurrencia)

T1 | T2 | T3 | T4 | T1 | T2 | T3 | T4 | ...

Los hilos se intercalan en el tiempo; solo uno ejecuta por ciclo.

2.3 En múltiples núcleos (paralelismo real)

Varios hilos pueden ejecutarse simultáneamente en distintos núcleos.

2.4 Tipos de paralelismo

Tipo	Descripción	Ejemplo
Paralelismo de datos (Data parallelism)	Se distribuyen subconjuntos de los mismos datos entre núcleos, y todos ejecutan la misma operación.	Procesar 1 millón de píxeles en 4 núcleos (250k cada uno).
Paralelismo de tareas (Task parallelism)	Se distribuyen hilos diferentes en los núcleos, cada uno realizando una operación distinta.	Un hilo procesa audio, otro video, otro red.

3: Desafíos para el desarrollador en sistemas multicore

El documento enumera los siguientes desafíos:

1. Dividir actividades (Dividing activities): Identificar qué partes del programa pueden ejecutarse en paralelo.
2. Balance (Balance): Distribuir la carga de trabajo equitativamente entre núcleos (evitar que un núcleo trabaje mucho y otro esté ocioso).
3. División de los datos (Data splitting): Partir los datos para que cada hilo tenga su porción (evitando contención).
4. Dependencia de datos (Data dependency): Identificar y sincronizar accesos a datos compartidos (condiciones de carrera).
5. Testing y debugging: Los errores de concurrencia (deadlocks, race conditions) son difíciles de reproducir y depurar.

4: Hilos de Usuario vs. Hilos de Kernel

4.1 Hilos de usuario (User-level threads)

- Gestión: Se manejan completamente en el espacio de usuario mediante una librería (ej: Pthreads en modo usuario antiguo, GNU Portable Threads).
- Ventajas: Muy rápidos (no hay llamadas al sistema para crear, sincronizar o destruir hilos).

- Desventajas: Si un hilo se bloquea en una llamada al sistema, se bloquea todo el proceso. Además, no pueden aprovechar múltiples núcleos (el kernel ve un solo hilo).

4.2 Hilos de kernel (Kernel-level threads)

- Gestión: El kernel conoce y maneja cada hilo individualmente.
- Ventajas: Pueden ejecutarse en paralelo en distintos núcleos; el bloqueo de un hilo no bloquea a los demás.
- Desventajas: Mayor overhead (cada operación de hilo requiere un cambio al modo kernel).

4.3 Implementación en Linux

- En Linux, los hilos se implementan como tareas (tasks) que comparten recursos.
- Se pueden ver los hilos de kernel con:
 - bash
 - `ps --ppid 2 -p 2 -o uname,pid,ppid,cmd,cls`
 - `kthreadd` (PID 2) es el proceso padre de todos los hilos de kernel.
 - Los hilos del kernel tienen nombres entre corchetes: `[ksoftirqd/0]`, `[kworker/0:0H]`, `[rcu_sched]`, etc.
- Para ver los hilos de un proceso específico (dado su PID):
 - bash
 - `ps -T -p <pid>`

5: Modelos de Relación entre Hilos de Usuario y Kernel

Existen tres modelos clásicos para mapear hilos de usuario a hilos de kernel.

5.1 Modelo Muchos a Uno (Many-to-One)

- Muchos hilos de usuario se mapean a un solo hilo de kernel.
- El kernel ve un único hilo.
- Problema: No pueden ejecutarse en paralelo en multicore (solo un hilo de usuario puede estar activo a la vez dentro del kernel).
- Ejemplos históricos: Solaris Green Threads, GNU Portable Threads (obsoletos para paralelismo real).

5.2 Modelo Uno a Uno (One-to-One)

- Cada hilo de usuario se mapea a un hilo de kernel distinto.
- Ventaja: Máximo paralelismo posible.
- Desventaja: Overhead por hilo (el kernel debe gestionar cada uno). Puede haber limitación en la cantidad de hilos por proceso.
- Ejemplos: Windows (threads nativos), Linux (NPTL – Native POSIX Thread Library), Solaris 9 y posteriores.

5.3 Modelo Muchos a Muchos (Many-to-Many)

- Muchos hilos de usuario se multiplexan sobre muchos hilos de kernel (pero menos que la cantidad de hilos de usuario).
- El kernel crea una cantidad adecuada de hilos de kernel (por ejemplo, igual al número de núcleos).
- Ventaja: Buena concurrencia sin el overhead de un hilo de kernel por cada hilo de usuario.
- Ejemplos: Solaris 9 (con modelo "dos niveles" anterior), Windows con Thread Fibers (emulación de hilos de usuario sobre hilos de kernel).

5.4 Tabla comparativa

Modelo	Relación	Paralelismo real	Overhead	Ejemplo	
Muchos a uno	N:1	No	Bajo	GNU Pth	
Uno a uno	1:1	Sí	Medio/Alto	Linux Windows	NPTL, (antiguo),
Muchos a muchos	N:M	Sí	Medio	Solaris Fibers	

6: Librerías de Hilos (Thread Libraries)

Son APIs que permiten al desarrollador crear y manejar hilos.

6.1 Principales librerías

- POSIX Pthreads (IEEE 1003.1c): Estándar en sistemas UNIX (Linux, Solaris, macOS, BSD). Especifica el comportamiento, no la implementación (puede ser a nivel usuario o kernel según el SO).
- Windows threads: API nativa de Microsoft.
- Java threads: Integrada en la JVM; normalmente mapea a hilos del SO anfitrión.

6.2 Implementación de librerías

- Totalmente en espacio de usuario: Ejemplo: antiguas versiones de Pthreads (LinuxThreads). Rápidas pero no aprovechan múltiples núcleos.
- A nivel de kernel (soportadas por el SO): Ejemplo: NPTL en Linux moderno. Cada hilo de usuario es un hijo del kernel.

7: Ejemplo Práctico en Pthreads (Linux)

El documento incluye un ejemplo clásico: calcular la suma de los primeros N números naturales usando un hilo.

7.1 Código completo comentado

Código C

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int sum;    // variable compartida entre el hilo principal y el secundario

// Función que ejecutará el hilo secundario
void *function(void *param);

int main(int argc, char *argv[]) {
    pthread_t tid;          // identificador del hilo
    pthread_attr_t attr;    // atributos del hilo (por defecto)
    int parametro_funcion = 4;

    // Inicializar atributos por defecto
    pthread_attr_init(&attr);

    // Crear el hilo. La función 'function' se ejecutará en paralelo.
    // El cuarto argumento es un puntero genérico (void*). Se castea el
    entero.
    pthread_create(&tid,      &attr,      function,      (void
    *) (intptr_t) parametro_funcion);

    // Esperar a que el hilo termine (join)
    pthread_join(tid, NULL);

    printf("Suma es = %d\n", sum);
    return 0;
}

// Función del hilo
void *function(void *param) {
    int i;
    int upper = (intptr_t) param;    // recuperar el parámetro (entero)
    sum = 0;
    for (i = 1; i <= upper; i++) {
        sum += i;
    }
}
```

```
    pthread_exit(0); // terminar el hilo (opcional, al salir de la
función también termina)
}
```

7.2 Compilación y ejecución

En Linux, compilar con:

```
bash
gcc -pthread programa.c -o programa
./programa
```

7.3 Múltiples hilos (espera con join)

Para crear varios hilos y esperarlos a todos:

Código C

```
#define NUM_THREADS 10
pthread_t workers[NUM_THREADS];

// Crear los hilos...
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_create(&workers[i], &attr, function, (void*)(intptr_t)i);
}

// Esperar a todos
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(workers[i], NULL);
}
```

8: clone() – La llamada al sistema subyacente en Linux

- En Linux, pthread_create() internamente utiliza la llamada al sistema clone().
- clone() permite crear una nueva tarea (hijo) que puede compartir selectivamente recursos con el padre, mediante flags.

8.1 Flags principales de clone()

Flag	Significado
CLONE_FS	Comparte información del sistema de archivos (raíz, cwd).
CLONE_VM	Comparte el mismo espacio de memoria (esencial para hilos).

CLONE_SIGHAND	Comparte manejadores de señales.
CLONE_FILES	Comparte la tabla de archivos abiertos.

8.2 Analogía con procesos tradicionales (fork() vs. clone())

- fork() crea un proceso hijo sin compartir memoria (copia en escritura, pero no comparte).
- clone() con CLONE_VM crea un hilo (comparte memoria).

9: Resumen de modelos de multithreading

Modelo	Descripción	Uso en la actualidad
Many-to-one	Múltiples hilos usuario → 1 hilo kernel	Obsoleto para paralelismo real
One-to-one	Cada hilo usuario → 1 hilo kernel	Estándar (Linux, NPTL, Windows)
Many-to-many	Múltiples hilos usuario → M hilos kernel ($M < N$)	Usado en sistemas con muchos hilos ligeros (Fibers)

El uso de hilos (threads) es esencial para aprovechar los procesadores multicore modernos. Los sistemas operativos actuales (Linux, Windows, macOS) implementan el modelo uno a uno (1:1) con librerías como Pthreads (en UNIX) o los threads nativos de Windows. La programación multihilo requiere considerar cuidadosamente la sincronización, el balance de carga y la división de datos para evitar condiciones de carrera y deadlocks.

Comprender la diferencia entre hilos de usuario y hilos de kernel, así como los modelos de mapeo, es fundamental para escribir software concurrente eficiente y portable.